TRABAJO PRÁCTICO · AVANCE TP4 (SP4 Y SP5, PRIMERA PARTE)

Integración con JavaScript

Caso de estudio: Nutrite — Dietética Online

Materia	Aplicaciones Web I — Sistemas
Profesor	Andrés Senn
Alumnos	Joaquín Baigorria Giuliano Baldini Matías Davico Franco Tellechea
Fecha	Noviembre de 2026 — Córdoba
Repositorio	github.com/Jbaigorria22/aw1-tp (rama `main`)
Entrega	AVANCE — primera parte

1. Introducción y alcance

En el SP3 (estilización con CSS) Nutrite quedó maquetado en HTML5 y estilizado con CSS, sin JavaScript. En esta primera parte del TP4 se incorpora JavaScript (módulos ES, `fetch`, `localStorage` y eventos del DOM) para que el sitio deje de ser estático, y se completan las vistas que la propuesta planteaba, en particular el proceso de compra por pasos.

Requisito de la consigna	Qué se hizo	Estado
Catálogo renderizado vía JavaScript	La grilla se dibuja con `render.js` a partir de los datos de `productos.json` (sección 2).	Hecho
Datos del catálogo en un módulo independiente	`datos.js` + `assets/data/productos.json`. Ninguna otra parte del código sabe de dónde salen los productos (sección 2.1).	Hecho
Organizar y modularizar el JavaScript	Un módulo por responsabilidad y un punto de entrada por página (sección 4).	Hecho
Eventos JavaScript	Filtros, buscador, orden, carrito, pasos del checkout y validaciones (sección 5).	Hecho
Terminar las vistas planteadas	Checkout de 4 pasos, ficha de producto dinámica, inicio dinámico, contacto con validación y blog (sección 3).	Hecho
Informe PDF con cambios respecto de la propuesta	Este documento, en el mismo PDF del proyecto (sección 6).	Hecho

Trabajo en equipo

Se trabajó con un repositorio único en GitHub, una rama por tema y Pull Requests revisados antes de integrar a `main`.

Integrante	Aporte principal
Joaquín Baigorria	Base del TP4 (sin `pages/` y con CSS unificado), carga del catálogo con `fetch` desde JSON, módulos `datos.js`, `render.js` y `filtros.js`, y eventos de filtros, buscador y orden. Revisión de los Pull Requests del equipo.
Giuliano Baldini	Checkout por pasos, ficha de producto dinámica, inicio, contacto con validación y blog; módulo `agregar.js`.
Matías Davico	Lógica del carrito y del checkout: `funciones-carrito.js`, `carrito.js` y la primera versión de `checkout.js`.
Franco Tellechea	Armado del informe en PDF y su entrega en el repositorio.

Cómo ver el proyecto

Como se usan módulos ES y `fetch`, el sitio debe abrirse con un servidor local y no con doble clic sobre el HTML. Por ejemplo, desde la carpeta del proyecto: `python -m http.server 8000` y abrir `http://localhost:8000/index.html`, o la extensión Live Server de VS Code.

2. Catálogo: datos, renderizado y filtros

En el SP3 las ocho tarjetas de producto estaban escritas a mano en el HTML. Ahora el HTML solo contiene la estructura (filtros, barra de orden y un contenedor vacío) y el JavaScript dibuja las tarjetas.

2.1 Estructura de datos en un módulo independiente

Los productos viven en `assets/data/productos.json`, un arreglo de objetos, y se leen únicamente desde `datos.js`. Si mañana los datos vinieran de una API, solo cambiaría ese archivo.

```
// assets/js/datos.js
export async function obtenerProductos() {
  const respuesta = await fetch("assets/data/productos.json");
  if (!respuesta.ok) throw new Error(`Error al cargar productos: ${respuesta.status}`);
  return respuesta.json();
}
```

Campo	Tipo	Uso
`id`	número	Identifica al producto en el carrito y en la URL de la ficha (`producto.html?id=2`).
`nombre`, `categoria`	texto	Título de la tarjeta y filtro por categoría.
`imagen`	texto (ruta)	Ilustración SVG del producto.
`badges`	arreglo de texto	Atributos (Sin TACC, Vegano, Sin azúcar, Integral): etiquetas de color y filtro por atributo.
`stock`	objeto `{estado, texto}`	Estado `disponible`, `ultimas` o `sin-stock`; define el semáforo y si el botón de compra está habilitado.
`precio`, `cuotas`	número	Precio y valor de cada cuota sin interés. Se formatean al dibujar la tarjeta.
`descripcion`, `ingredientes`	texto	Contenido de la ficha de producto.

2.2 Renderizado con JavaScript

`render.js` convierte un producto en una tarjeta (`crearTarjetaProducto`) y dibuja una lista en el contenedor (`renderizarProductos`). Si no hay resultados muestra “No hay productos que coincidan con los filtros” y si falla la carga muestra un mensaje de error en lugar de dejar la pantalla vacía. La misma tarjeta se reutiliza en el inicio, el catálogo, la ficha y el blog.

2.3 Filtros, búsqueda y orden

`filtros.js` contiene funciones puras (reciben datos y devuelven datos, sin tocar el DOM), por lo que se pueden probar solas:

- **Atributos:** el producto debe tener *todos* los atributos tildados (condición Y). Con “Sin TACC” y “Vegano” se ven solo los productos que cumplen ambos.
- **Categoría:** se muestran los productos de cualquiera de las categorías tildadas.
- **Solo con stock:** oculta los productos en estado `sin-stock`.

- **Buscador:** coincidencia parcial sobre el nombre.
- **Orden:** Más vendidos (orden original), Menor precio, Mayor precio y Nombre A-Z.

El contador “Mostrando X de Y productos” se calcula con los datos reales; se eliminaron los números inventados que tenía el maquetado del SP3.

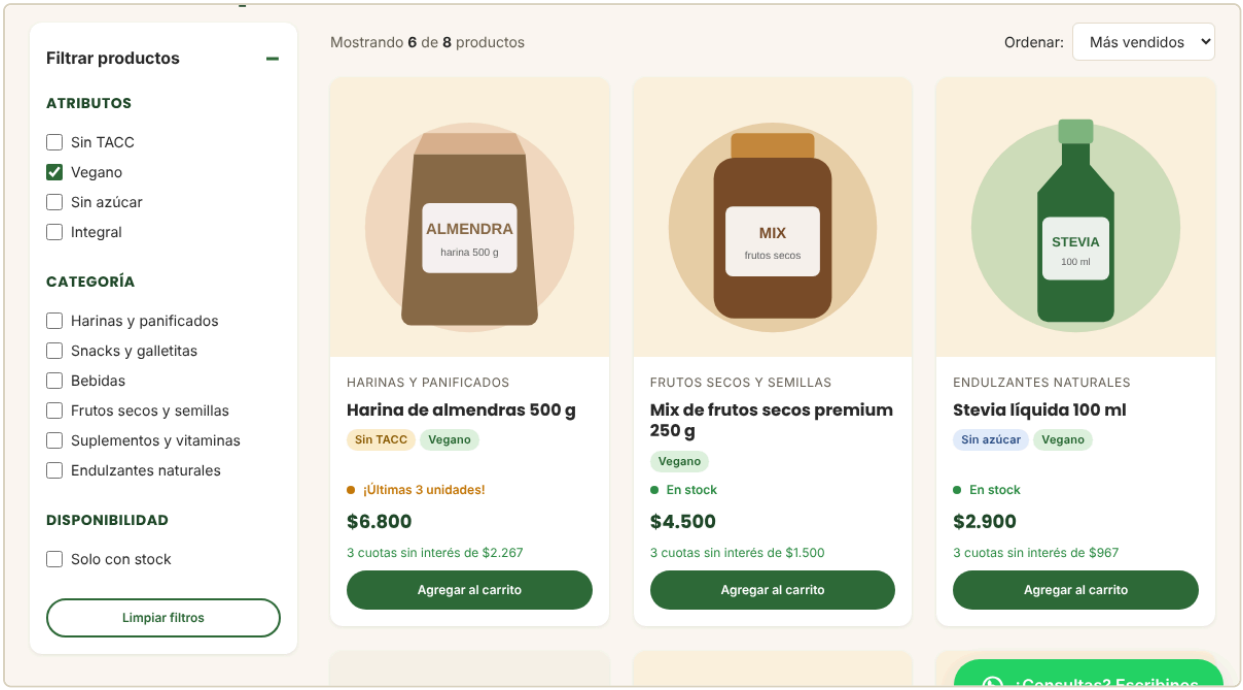


Figura 1 — Catálogo renderizado por JavaScript con el filtro “Vegano” aplicado (6 de 8 productos).

> **Limitación conocida:** la búsqueda distingue acentos (“chia” no encuentra “chía”). Se dejó así para esta entrega; normalizar el texto antes de comparar es una mejora sencilla para la segunda parte.

3. Vistas desarrolladas

Vista	Archivo	Novedad en el TP4
Inicio	`index.html`	Los productos destacados se dibujan con JS (`inicio.js`) y el contador del carrito refleja el estado real.
Catálogo	`catalogo.html`	Renderizado por JS, filtros, buscador, orden, "Agregar al carrito" y búsqueda desde el header (`?q=`).
Ficha de producto	`producto.html`	Dinámica: lee `?id=` y muestra el producto elegido (antes era un único producto fijo).
Carrito	`carrito.html`	Cantidades, eliminación, envío y totales con JS.
Checkout	`checkout.html`	Los 4 pasos del stepper: Datos, Entrega, Pago y Confirmación (antes solo existía el paso Pago).
Contacto	`contacto.html`	Validación del formulario con mensajes de error y confirmación.
Blog	`blog.html`, `blog-articulo.html`	Sección nueva: listado filtrable y vista de cada artículo con productos relacionados.

3.1 Carrito

El carrito se guarda en `localStorage` (clave `carrito`, arreglo de `{id, cantidad}`) y todas las páginas lo leen mediante `funciones-carrito.js`. En `carrito.html` se puede cambiar la cantidad (con tope en el stock), eliminar productos, elegir el tipo de envío y ver el total, con el envío gratis a partir de $40.000.

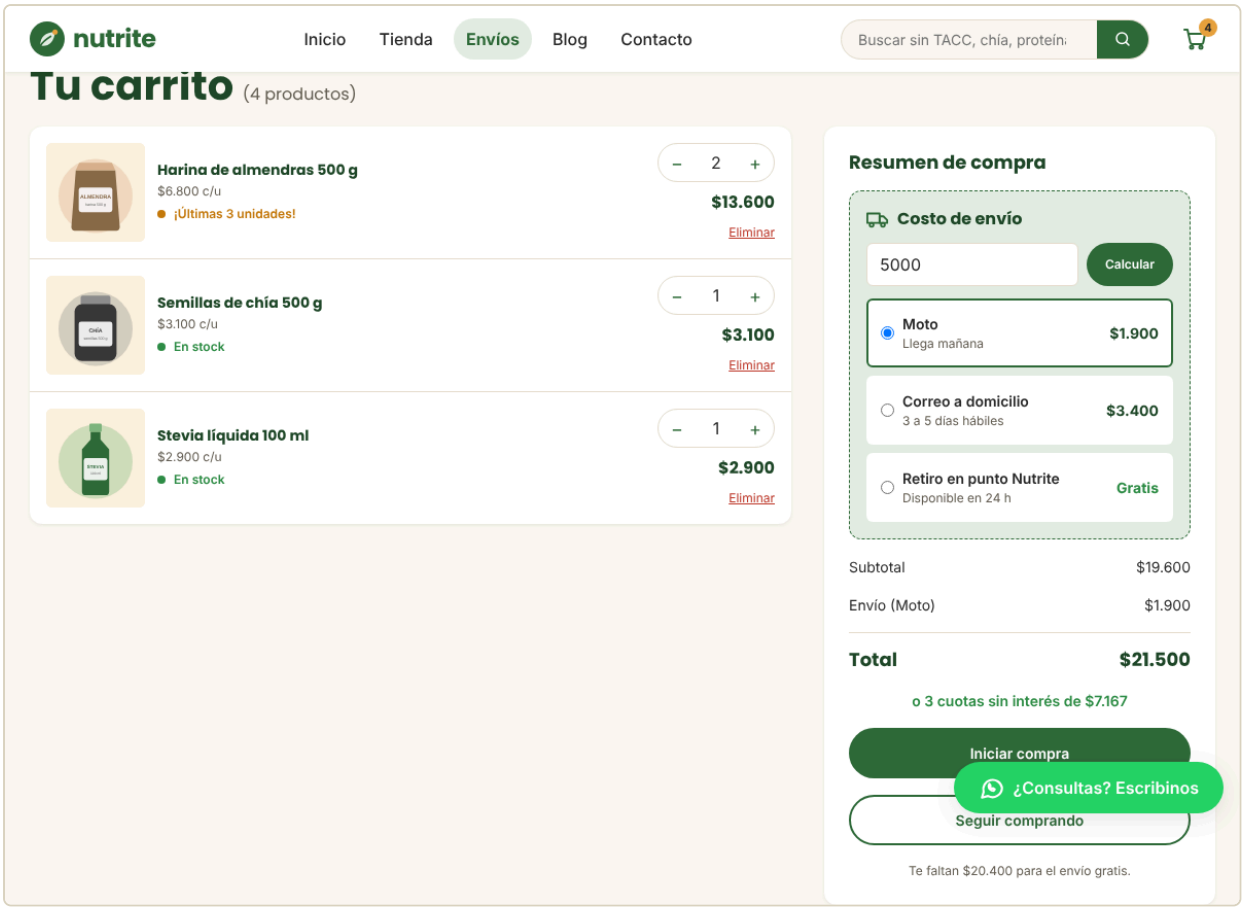


Figura 2 — Carrito con cantidades editables, envío y resumen.

3.2 Checkout por pasos

El proceso de compra planteado en la propuesta (stepper con 4 etapas) quedó completo en una sola página: `checkout.html` contiene un panel por paso y `checkout.js` muestra uno solo por vez, actualizando el stepper (paso activo, pasos completados y `aria-current`).

Paso	Contenido	Validaciones
1. Datos	Nombre y apellido, email, teléfono, DNI.	Nombre ≥ 3 caracteres; email con formato válido; teléfono de 8 a 12 números; DNI de 7 u 8 números.
2. Entrega	Moto, correo o retiro en punto Nutrite; dirección (calle, ciudad, CP, aclaraciones).	Si el envío es “Retiro” se oculta la dirección; si no, calle, ciudad y CP de 4 números son obligatorios.
3. Pago	Crédito (con tabla de cuotas), débito, transferencia (10% OFF) y billetera.	Total y cuotas se recalculan al cambiar de medio o de plan.
4. Confirmación	Número de pedido (`NUT-######`), datos de entrega, medio de pago y total pagado. Vacía el carrito.	—

El resumen del pedido (columna lateral) se actualiza en tiempo real con el tipo de envío y el medio de pago elegidos. Cada paso permite volver al anterior sin perder lo cargado. Los datos del cliente se escapan antes de insertarse en el HTML de la confirmación para evitar inyección de código.

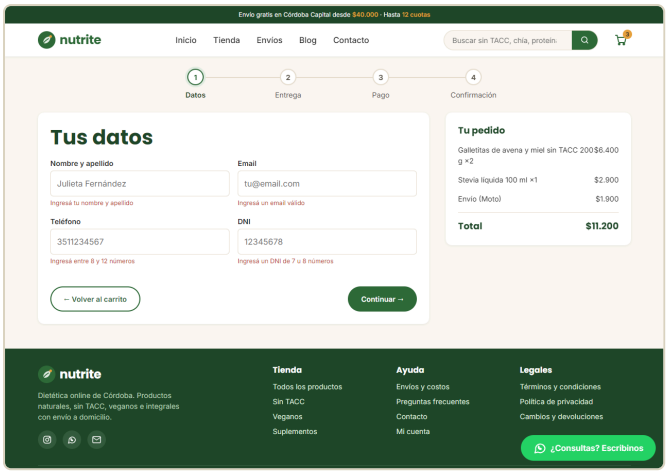


Figura 3 — Paso 1 (Datos) con validación de campos.

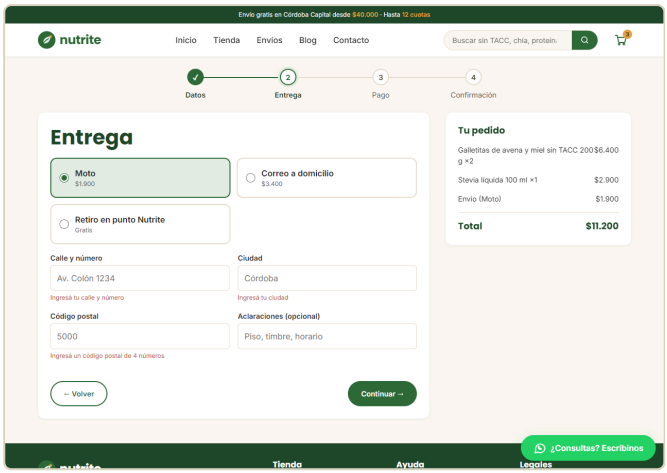


Figura 4 — Paso 2 (Entrega) con resumen lateral.

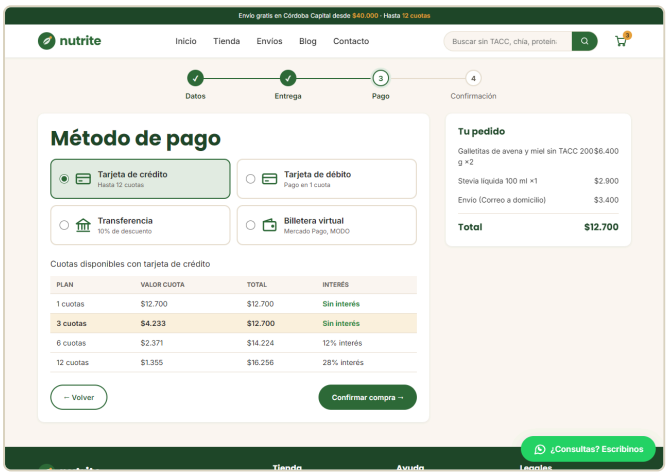


Figura 5 — Paso 3 (Pago): medios de pago y cuotas.

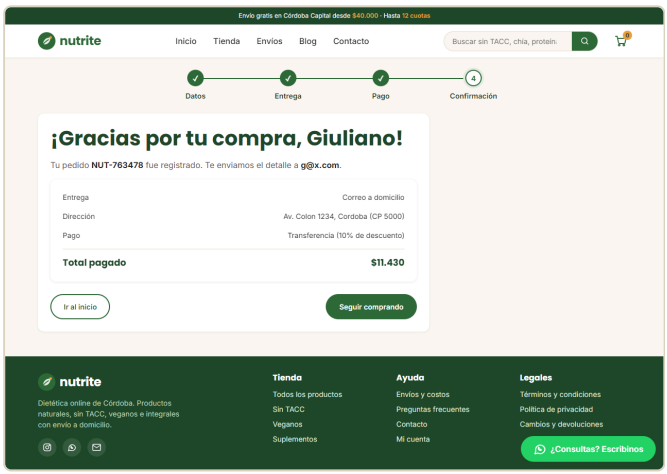


Figura 6 — Paso 4 (Confirmación) con número de pedido.

3.3 Ficha de producto dinámica

`producto.js` lee el parámetro `id` de la URL (`producto.html?id=2`), busca el producto en el módulo de datos y dibuja la ficha: galería con miniaturas, badges, stock, precio, cuotas, precio con transferencia, selector de cantidad (limitado al stock), cálculo de envío por código postal, botón de WhatsApp con el nombre del producto en el mensaje y acordeones de descripción e ingredientes. Los productos relacionados (misma categoría primero) se dibujan con la misma tarjeta del catálogo. Si el `id` no existe se muestra un mensaje y, si el producto no tiene stock, el botón de compra queda deshabilitado.

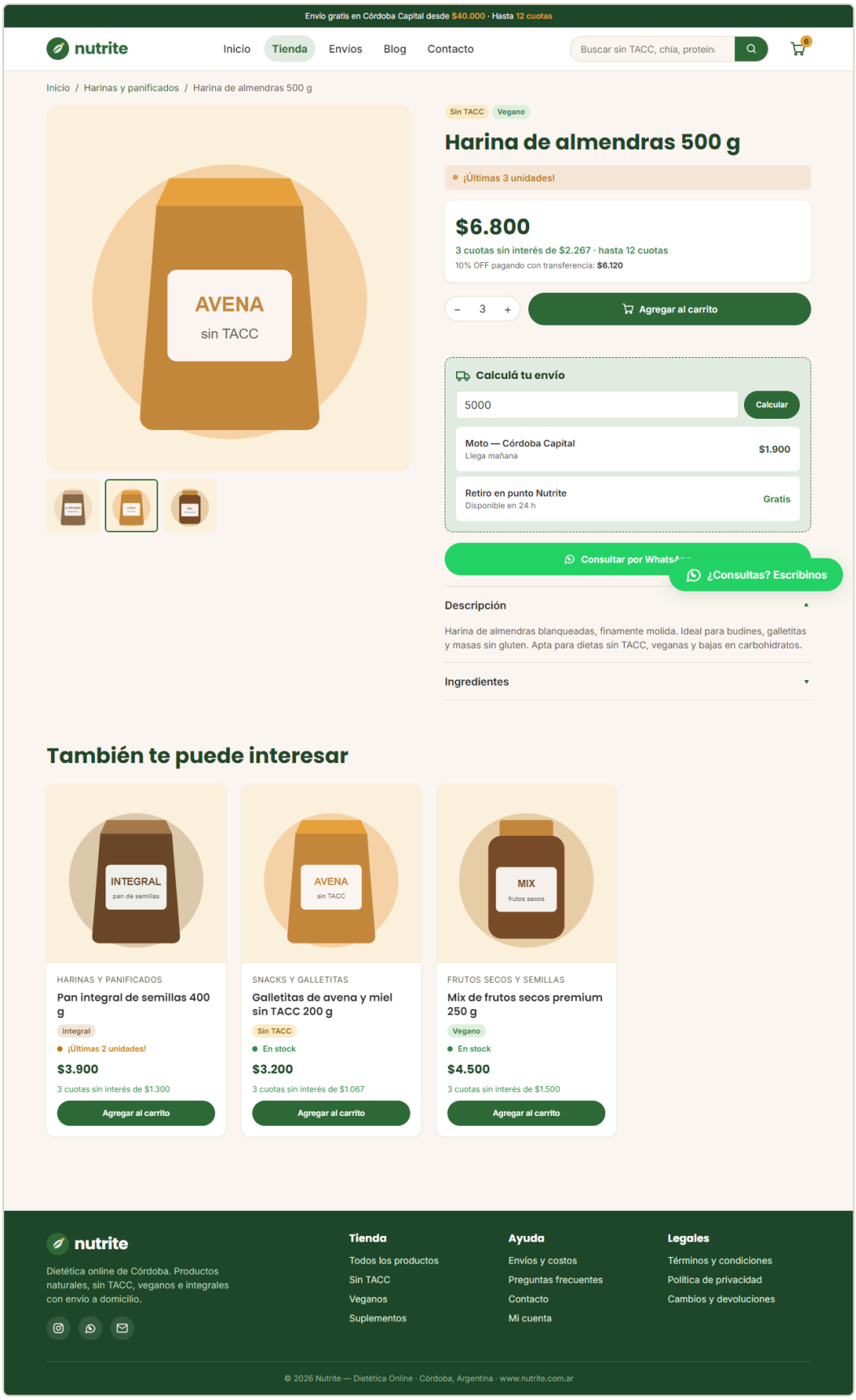


Figura 7 — Ficha de producto generada por JavaScript.

3.4 Inicio y contacto

- **Inicio:** los 4 primeros productos con stock se dibujan desde el mismo JSON, reutilizando `crearTarjetaProducto`.

- **Catálogo:** las tarjetas enlazan a `producto.html?id=N` y el botón “Agregar al carrito” suma al carrito y muestra “¡Agregado!”. El buscador del header de cualquier página llega a `catalogo.html?q=texto` y filtra al abrir.
- **Contacto:** validación de nombre, email y mensaje (≥ 10 caracteres) con marcado de errores (`campo--error`) y mensaje de confirmación.

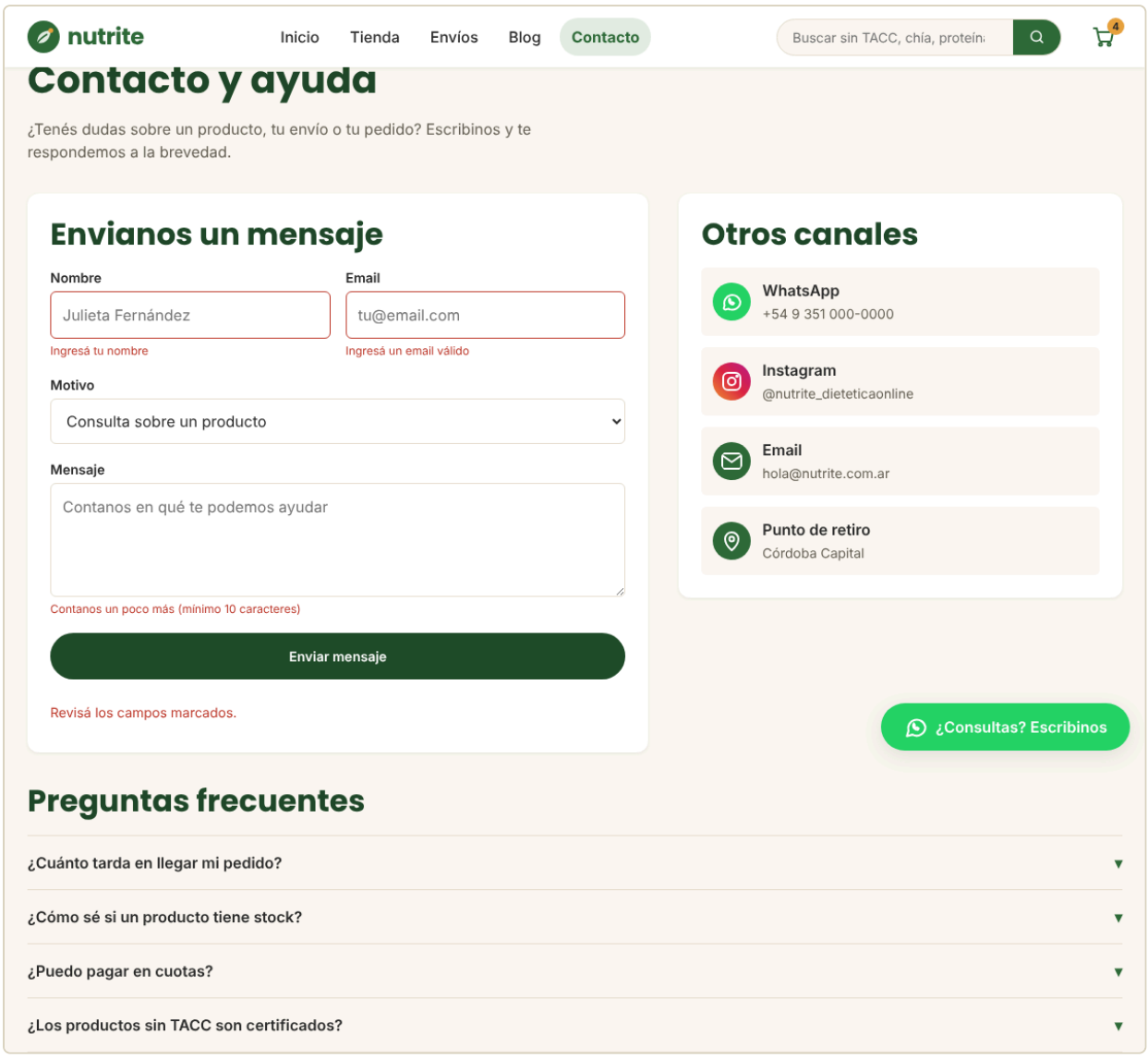


Figura 8 — Formulario de contacto con validación.

3.5 Blog

Se agregó una sección de blog simple, enlazada desde el menú “Blog” de todas las páginas (antes el enlace era un marcador `#`). Contiene 6 artículos de ejemplo (recetas, tips y novedades) guardados en `assets/data/articulos.json`, con título, categoría, fecha, resumen, texto y los ids de los productos relacionados.

- **Listado** (`blog.html` + `blog.js`): tarjetas dibujadas por JS, chips de categoría (Todos / Recetas / Tips / Novedades), buscador por título con filtrado al escribir y contador de resultados.
- **Artículo** (`blog-articulo.html?id=N` + `articulo.js`): lee el id de la URL, muestra el texto y, al final, tarjetas de los productos del catálogo mencionados, con “Agregar al carrito”.
- **Módulos:** `datos.js` suma `obtenerArticulos()` y `render-blog.js` dibuja tarjetas y artículo, con la misma separación datos / render / punto de entrada del catálogo.

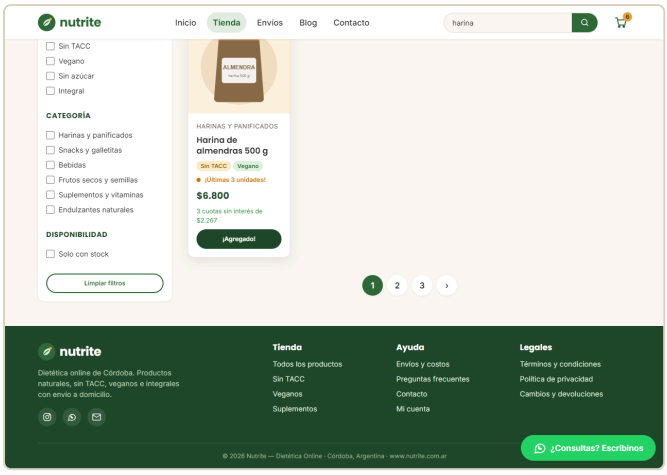


Figura 9 — Listado del blog con filtros y buscador.

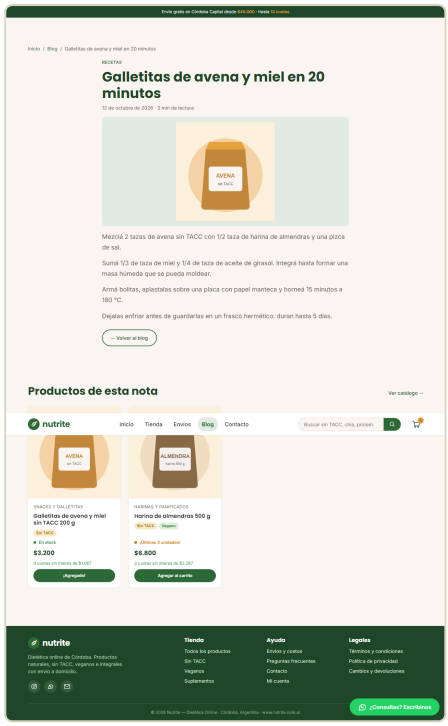


Figura 10 — Artículo con productos relacionados.

4. Organización y modularización del JavaScript

Todo el código vive en `assets/js/` y usa módulos ES (`<script type="module">`). Cada archivo tiene una sola responsabilidad; cada página tiene un archivo "punto de entrada" que coordina los módulos.

Módulo	Responsabilidad	Usado por
`datos.js`	Único lugar que sabe de dónde salen los datos (`fetch` a los JSON).	Todas las páginas con productos o artículos
`render.js`	Convierte un producto en una tarjeta HTML y formatea precios.	Catálogo, inicio, producto, artículo
`filtros.js`	Funciones puras de filtrado y orden.	Catálogo
`funciones-carrito.js`	Carrito en `localStorage`, totales, envíos y cuotas.	Carrito, checkout, agregar
`agregar.js`	Botones "Agregar al carrito" (delegación de eventos) y contador del header.	Catálogo, inicio, producto, artículo, blog, contacto
`render-blog.js`	Dibuja tarjetas y detalle de los artículos.	Blog, artículo
`catalogo.js`, `producto.js`, `inicio.js`, `contacto.js`, `carrito.js`, `checkout.js`, `blog.js`, `articulo.js`	Puntos de entrada: buscan los elementos del DOM, piden los datos, llaman a los módulos y registran los eventos.	Una página cada uno

```
productos.json → datos.js → ( render.js , filtros.js ) → catalogo.js / inicio.js / producto.js
localStorage  ⇄ funciones-carrito.js → carrito.js / checkout.js / agregar.js
```

Un caso concreto de la ventaja de separar: el filtrado (`filtros.js`) no sabe nada del DOM y el dibujo (`render.js`) no sabe nada de los filtros. Por eso la tarjeta del catálogo se reutilizó sin cambios en el inicio, la ficha y el blog.

5. Eventos JavaScript

Evento	Elemento	Efecto
`change`	Checkboxes de filtros, select de orden	Vuelve a dibujar el catálogo
`submit`	Buscador	Filtra por nombre sin recargar la página
`reset`	"Limpiar filtros"	Restaura el catálogo completo
`click` (delegado)	"Agregar al carrito"	Suma al carrito, actualiza el contador y muestra "¡Agregado!". Un solo *listener* sirve a todas las tarjetas, incluso las que se dibujan después de filtrar

Evento	Elemento	Efecto
`click`	Miniaturas de la ficha	Cambia la imagen principal
`click` / `change`	Cantidad (− / +)	Ajusta la cantidad con tope en el stock
`submit`	Calculadora de envío (ficha)	Valida el CP y muestra opciones
`submit`	Formularios de Datos y Entrega	Valida, guarda y avanza de paso
`click`	“Volver” del checkout	Retrocede de paso
`change`	Opciones de envío y medios de pago	Recalcula el resumen y las cuotas
`click`	Fila de cuotas, “Confirmar compra”	Elige el plan; genera la confirmación
`click` / `input`	Chips y buscador del blog	Filtra los artículos por categoría y título
`submit` / `input`	Formulario de contacto	Valida; limpia el error al corregir el campo

6. Cambios respecto de la propuesta

Cambios técnicos de organización

- **Un solo `main.css` en lugar de 19 archivos con `@import`.** En el SP3 cada página enlazaba `main.css`, que importaba 19 archivos parciales. Sin un empaquetador, el navegador descarga `main.css` y recién entonces descubre los 19 imports y los pide, es decir una cascada de pedidos antes de poder dibujar la página. Por esa razón, y por la devolución del profesor sobre el uso de `@import`, se unificaron en un único archivo respetando el orden de la cascada (base, layout, componentes, páginas, utilidades), con un comentario de sección por cada parte. *Contrapartida:* el archivo queda largo (unas 1.250 líneas). La alternativa evaluada fue enlazar varios archivos con `<link>` en cada HTML, que se descargan en paralelo, pero obliga a repetir los enlaces y respetar el orden en las 8 páginas; se priorizó un único pedido y una sola fuente de estilos.
- **Sin carpeta `pages/`.** Todos los HTML están en la raíz, junto a `index.html`. Las rutas pasaron de `../assets/…` a `assets/…` en todas las páginas, lo que simplifica los enlaces y evita errores de rutas relativas.
- **Datos fuera del HTML.** Las tarjetas de producto dejaron de estar escritas a mano y pasaron a `productos.json`. Se quitaron del maquetado los contadores y chips decorativos que no correspondían a datos reales.

Cambios funcionales

- **Checkout en una sola página:** la propuesta planteaba un proceso por pasos; en lugar de un HTML por paso, se resolvió con paneles dentro de `checkout.html` controlados por JS. Se evita recargar la página y se conservan los datos entre pasos.
- **Ficha de producto genérica:** pasó de un producto fijo a una plantilla alimentada por `?id=`. Se quitaron el precio anterior tachado y la tabla nutricional, porque el catálogo no tiene esos datos.
- **Persistencia:** al no haber backend, el carrito y el tipo de envío se guardan en `localStorage`. El pedido confirmado no se envía a ningún servidor: la confirmación es simulada y el número de pedido es aleatorio.
- **Blog:** no estaba detallado en la propuesta; se agregó como sección simple con contenido de ejemplo (sin backend ni panel de edición).
- **Contacto:** el formulario valida y confirma, pero no envía el mensaje (requiere backend, fuera del alcance de esta entrega).
- **Regla de visibilidad:** se agregó `[hidden] { display: none; }` al CSS para que el atributo `hidden` funcione sobre componentes con `display: grid`.

Limitaciones conocidas

- El stock máximo por producto se calcula a partir del texto de stock (“Últimas N unidades”; 10 en el resto), porque el JSON no tiene un campo numérico de stock.
- El carrito parte de un ejemplo precargado la primera vez que se abre el sitio (decisión del módulo de carrito).
- La búsqueda del catálogo distingue acentos.
- La paginación del catálogo y los enlaces de “Mi cuenta”, legales y redes sociales siguen como marcadores: quedan para la segunda parte.

7. Verificación

Se probaron los flujos en Chrome con un script automatizado (Playwright): catálogo con búsqueda por URL, filtros, orden y limpieza de filtros; blog (navegación, filtros, buscador, artículo y productos relacionados); agregar al carrito y contador entre páginas; ficha con `id` válido, inexistente y sin stock; validaciones y mensajes de contacto; recorrido completo del checkout (errores por paso, retiro y envío, volver atrás, transferencia, confirmación y carrito vaciado); ausencia de scroll horizontal en 390 px y de errores de consola. Cada rama se integró a `main` mediante Pull Request y se volvió a probar el resultado completo sobre `main`.

Anexo — Uso de Inteligencia Artificial

El grupo utilizó Claude (Anthropic) como asistente para entender y verificar el código, y como apoyo en el desarrollo y en la redacción de este informe. Su uso queda declarado acá y en el historial del repositorio, donde varios commits figuran con Claude como coautor.

Para qué se usó	Qué hizo el equipo
Entender cómo funcionan los módulos, los eventos y `fetch`, y cómo se conectan las páginas con el carrito.	Cada integrante leyó y probó el código de su parte antes de integrarla a `main`.
Verificar y revisar el código del equipo (Pull Requests) y probar los flujos de punta a punta.	Se probaron filtros, buscador, orden, carrito, checkout, contacto y blog, y se corrigieron los problemas encontrados (por ejemplo, la búsqueda que distingue acentos quedó documentada como limitación).
Ordenar la base del proyecto (quitar `pages/`, unificar los CSS, corregir rutas).	Se comprobó visualmente cada página y que no quedaran rutas rotas.
Redactar y unificar este informe.	Se comprobó que lo declarado coincidiera con lo que hay en `main`.

Las ilustraciones de productos son SVG propios usados como marcador de posición; precios, número de WhatsApp, dirección y artículos del blog son datos de ejemplo.